DEVOPS WINTER ARC

HANDS-ON ASSIGNMENT

DAY 15 | Kubernetes Fundamentals

90 Days Challenge | Week 3: Kubernetes | Learn • Deploy • Scale • Inspect

LEVEL	TIME	FOCUS
Beginner / Advanced beginner	3–4 hours	Kubernetes architecture, Pods, Deployments, Services, YAML manifests, kubectl troubleshooting

GOAL OF THE DAY

Understand how Kubernetes runs containerized applications, then deploy an Nginx application to a local Kubernetes cluster using YAML manifests. Verify the workload, scale it, expose it with a Service, and document the evidence.

YOUR TASKS TODAY

- Explain the purpose of Kubernetes and the difference between Docker and Kubernetes.
- Identify a cluster, control plane, worker node, Pod, Deployment, and Service.
- Set up or verify a local cluster using Docker Desktop Kubernetes or kind.
- Create YAML manifests for a Deployment and a NodePort Service.
- Deploy Nginx, inspect Pods, scale replicas to three, and verify the Service.
- Practice basic troubleshooting and submit screenshots plus a README.

DELIVERABLE

A GitHub repository or folder named **day15-kubernetes/** containing **deployment.yaml**, **service.yaml**, **README.md**, and an **evidence/** folder with screenshots of the running cluster, Pods, Deployment, scaling, and Service.

> **Safety / cost note:** This assignment is designed for a local cluster. Do not create an AWS EKS cluster for this lab unless you understand its costs and have approval. Never upload credentials or kubeconfig files to GitHub.

01 / UNDERSTAND KUBERNETES

Kubernetes is a container orchestration platform. It helps deploy and manage containerized applications across a cluster. You declare the desired state in manifests; Kubernetes controllers work to maintain that state.

CONCEPT	WHAT IT DOES	TODAY'S EXAMPLE
Cluster	The complete Kubernetes environment.	Your local cluster
Control plane	Makes cluster-level decisions and manages state.	Coordinates desired state
Worker node	Runs Pods and application workloads.	Runs Nginx Pod(s)
Pod	Smallest deployable unit; contains one or more containers.	Nginx container in a Pod
Deployment	Manages replicated Pods and supports rolling updates.	Keeps three Nginx replicas
Service	Provides a stable way to reach a set of Pods.	Routes traffic to Nginx Pods

Docker vs Kubernetes

- **Docker** builds and runs containers.
- **Kubernetes** orchestrates containerized workloads, including scheduling, scaling, and service discovery.

Commands to know

```
kubectl version --client
kubectl cluster-info
kubectl get nodes
kubectl get pods -A
kubectl get deployments
kubectl get services
```

If `kubectl` is not installed or no cluster is available, install Docker Desktop with Kubernetes enabled or use kind. Follow the official setup guide for your OS.

Official references: Kubernetes Concepts • kind Quick Start

02 / PREPARE YOUR LOCAL CLUSTER

Option A — Docker Desktop Kubernetes

- Open Docker Desktop and enable Kubernetes in Settings, if it is available in your installation.
- Wait until the cluster reports that it is running.
- Run `kubectl get nodes` and confirm the node status is `Ready`.

Option B — kind (Kubernetes in Docker)

```
# Install kind and kubectl first, following the official docs
kind create cluster --name winter-arc
kubectl cluster-info
kubectl get nodes
```

Use only one option. If a cluster already works, do not create another one unnecessarily.

03 / DEPLOY NGINX USING YAML

Create this folder structure:

```
day15-kubernetes/
■■■ deployment.yaml
■■■ service.yaml
■■■ README.md
■■■ evidence/
```

Step 1 — Create `deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

Step 2 — Create `service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Step 3 — Apply the manifests

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get deployments
kubectl get pods -o wide
kubectl get services
```

Expected result: the Deployment should become available, two Pods should reach `Running`, and the Service should show NodePort `30080`. Image tags and cluster setup may affect the time required.

Step 4 — Inspect the resources

```
kubectl describe deployment nginx-deployment
kubectl describe service nginx-service
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl logs deployment/nginx-deployment
```

If `kubectl logs deployment/nginx-deployment` is unsupported by your kubectl version or returns an ambiguity error, list the Pods with `kubectl get pods` and run `kubectl logs <pod-name>`.

04 / SCALE AND VERIFY

Step 5 — Scale the application to three replicas

```
kubectl scale deployment nginx-deployment --replicas=3
```

```
kubectl get deployment nginx-deployment
kubectl get pods
```

Confirm that three Pods are running or ready. The Deployment declares the desired replica count; Kubernetes attempts to maintain it.

Step 6 — Test access

- Try opening **http://localhost:30080** if your local Kubernetes setup exposes NodePort through localhost.
- If that address does not work in your environment, use port-forwarding as a reliable local test:

```
kubectl port-forward service/nginx-service 8080:80
# In another terminal, test:
curl http://localhost:8080
```

Stop port-forwarding with Ctrl+C after testing. Save a screenshot of the Nginx welcome page or successful curl output.

05 / TROUBLESHOOTING

PROBLEM	CHECK / FIX
kubectl cannot connect to server	Confirm Docker Desktop or kind cluster is running; check `kubectl config current-context`.
Pod is Pending	Run `kubectl describe pod ` and inspect Events and node availability.
Pod is ImagePullBackOff	Check the image name/tag and internet access; inspect the Pod events.
Pod is CrashLoopBackOff	Use `kubectl logs ` and `kubectl describe pod `.
Service has no endpoints	Check that Service selector labels match the Pod labels: `kubectl get pods --show-labels`.
Port 30080 does not open	Try `kubectl port-forward service/nginx-service 8080:80` and browse localhost:8080.

Step 7 — Demonstrate self-healing (optional)

```
kubectl get pods
kubectl delete pod <one-nginx-pod-name>
kubectl get pods -w
```

Watch Kubernetes create a replacement Pod because the Deployment is maintaining the desired replica count. Press Ctrl+C to stop watching.

06 / SUBMISSION CHECKLIST

- Repository/folder includes deployment.yaml, service.yaml, README.md, and evidence/.
- README explains Cluster, Pod, Deployment, Service, and Docker vs Kubernetes in your own words.
- Screenshots show `kubectl get nodes`, running Pods, Deployment, Service, and three replicas.
- Include proof of successful Nginx access using NodePort or port-forward.
- Include one troubleshooting command and describe what it helped you inspect.
- Do not upload kubeconfig files, tokens, cloud credentials, or other secrets.

Suggested README sections

```
# Day 15 - Kubernetes Fundamentals
## Objective
## Architecture and key concepts
## Cluster setup
## Deploy and scale Nginx
## Commands and evidence
## Troubleshooting / lessons learned
## Cleanup
```

Cleanup after taking evidence

```
kubectl delete -f service.yaml
kubectl delete -f deployment.yaml
# If you created a kind cluster only for this lab:
kind delete cluster --name winter-arc
```

07 / INTERVIEW READINESS

QUESTION	KEY IDEA
What is Kubernetes?	A platform for orchestrating containerized workloads across a cluster.
What is a Pod?	The smallest deployable Kubernetes unit; it contains one or more containers.
Why use a Deployment?	To manage replicated Pods and maintain the desired number of replicas.
What does a Service do?	Provides a stable endpoint to access a group of Pods.
What is a label selector?	A way for Kubernetes resources such as Services and Deployments to identify matching Pods.

QUESTION	KEY IDEA
What happens when a managed Pod is deleted?	The Deployment's controller normally creates a replacement to restore the desired state.
How is Docker different from Kubernetes?	Docker builds/runs containers; Kubernetes manages and orchestrates containerized workloads.

BONUS CHALLENGE — Change the Deployment from three replicas to five, verify the new Pods, and explain in your README how the Service continues routing to Pods selected by its labels.

REVIEW CRITERIA (100 POINTS)

AREA	POINTS
Cluster setup and Kubernetes concepts explained	20
Correct Deployment and Service YAML	25
Successful deployment and scaling to three replicas	25
Access test and troubleshooting evidence	15
README, screenshots, cleanup and security hygiene	15

FINAL DELIVERABLE: Submit your GitHub repository link and screenshots. Use actual results from your own cluster; do not fabricate successful outputs.